

Energy M&V Engine

Technical Documentation & User Guide

Version 9.6 • IPMVP Option C • Sigma Computing Python Element

Workbook: M&VEngine_MASTER

Contents

1. Overview
2. User Guide
3. Configuration Reference
4. Architecture & Processing Pipeline
5. Function Reference
6. Sigma Integration

1. Overview

The Energy M&V Engine is a measurement and verification (M&V) tool that quantifies energy savings at building sites by comparing actual metered consumption after an efficiency project against a statistical model of what consumption would have been without it. It implements the IPMVP Option C — Whole-Facility methodology, in which a weather-normalized regression model is fitted to historical (baseline) utility bills and then used to predict consumption during the post-project (reporting) period. The difference between predicted and actual consumption is the avoided usage, which the Engine also converts into avoided cost.

The Engine runs as a Python element inside the Sigma Computing workbook M&VEngine_MASTER. It reads billing data and (optionally) daily weather data directly from Sigma elements, performs all modeling in Python, and writes **eight result tables** back into the workbook for reporting and dashboarding. The same script can also be run locally against a CSV file for testing or offline analysis.

1.1 What the Engine does

- Cleans and normalizes raw utility billing data, including unit conversion to a common basis (kWh for electricity, therms for gas).
- Splits each meter's bills into a baseline period (before the commission date) and a reporting period (on or after it).
- Detects and removes outlier bills using statistical fences (IQR for baseline, percent-avoided fences for reporting).
- Searches for the best baseline model by evaluating many candidate combinations of baseline bills against seasonal-balance, weather-coverage, and statistical-quality criteria.
- Fits a weather-driven linear regression (HDD-only for gas; HDD + CDD for electricity) and grades it against four quality gates: **R²**, **CV-RMSE**, **weather coverage**, and **seasonal balance**.
- Where no combination of bills can satisfy the gates, still produces a best-effort model from the most recent bills so the site is reviewable rather than silently absent.
- Handles multi-meter sites, distinguishing concurrent meters (modeled individually and as a weighted aggregate) from sequential meter replacements (combined into one timeline).
- Computes avoided usage and avoided cost per bill and per model, and marks a single **Recommended** row per site/fuel so a portfolio total counts each site once.

1.2 Key concepts and terminology

Term	Meaning
Baseline period	Bills with a period start date before the meter's commission date. Used to train the model.
Reporting period	Bills with a period start date on or after the commission date. Where savings are measured.
HDD / CDD	Heating and Cooling Degree Days — weather measures that drive heating and cooling load. The regression inputs.
Avoided usage	Predicted consumption (from the baseline model) minus actual consumption during the reporting period.
R ²	Goodness of fit of the regression. Higher is better; thresholds differ for electricity and gas.
CV-RMSE	Coefficient of variation of the root-mean-square error. A normalized error measure; lower is better.
Weather coverage	How much of the full baseline weather range the selected baseline bills span. Guards against extrapolation. For gas it is measured on HDD only.
Seasonal balance	The weakest quarter's bill count ÷ the mean quarter's count (1.00 = perfectly even). Guards against a model leaning on one season. A quality gate.

Term	Meaning
Quarterly spread	$\text{std} \div \text{mean} \times 100$ of the four quarterly counts. Guards baseline shape; it is a selection filter and ranking term, not a pass/fail gate.
Idle quarter (gas)	A gas site's warmest quarter ($\text{HDD} \approx 0$), whose bills carry no heating-slope information. Derived per site and exempted from the seasonal rules.
Locked baseline	A pre-selected set of baseline bills reused on re-runs instead of re-optimizing. Marked in two columns: Individual Selected Baseline and Aggregated Selected Baseline .
Recommended	A boolean marking the one model per site/fuel that should be counted in savings — quality-passing and de-duplicated (an aggregate or its meters, never both).
Optimization Mode	How this run arrived at a baseline: Optimized, Most Recent, Locked, or Skipped.
About this document Sections 1–3 are oriented toward users and analysts running the Engine in Sigma. Sections 4–6 document the internal functions for developers and maintainers. The Engine version described here is v9.6 .	

2. User Guide

This section describes how to run the Engine, what inputs it expects, and how to read its outputs. It assumes you are operating the Engine inside the M&VEngine_MASTER Sigma workbook.

2.1 Inputs

The Engine reads its billing data from a Sigma element (default name **Input**). Each row is one utility bill for one meter. The columns below are recognized.

Required columns

Column	Description
Site Name	Identifier for the building/site. Groups bills and meters together.
Service Type	Determines whether a row is treated as electricity or gas (matched on “gas” / “elec”).
Period Start Date	Start of the billing period. Used for the baseline/reporting split and for seasonal balancing.
Period End Date	End of the billing period. Used to window daily weather into period totals.
Consumption	Metered usage for the period (converted to a common unit if Consumption Unit is supplied).
Total Charges	Dollar amount billed. Currency symbols and commas are stripped automatically.
Commission Date	Project go-live date. Bills before it are baseline; bills on/after are reporting.

Optional columns

Column	When used
WeatherStation	ICAO station ID. Required when the weather source is Sigma; joins bills to daily weather history.
HDD, CDD	Degree-day values. Required when the weather source is CSV; used as fallback when Sigma misses a row.
Meter Number	Distinguishes meters within a site. Drives multi-meter handling.
Consumption Unit	Unit label (e.g. kWh, therms, ccf, mcf, mmbtu) used to convert consumption to a common basis.
Individual Selected Baseline	Marks pre-chosen baseline bills for an individual meter with a value of 1, to lock that meter's baseline on re-runs.
Aggregated Selected Baseline	Marks pre-chosen baseline bills for a site/fuel aggregate. Kept separate so an individual and an aggregate lock at the same site don't collide.

Locking a baseline — two columns now

Baseline locks live in **Individual Selected Baseline** and **Aggregated Selected Baseline**. A legacy single **Selected Baseline** column is still accepted and read as the individual lock. An aggregate period is treated as locked only when **every pooled meter's row** in that period is flagged (the “all-meters” rule). The **input_with_baseline** output writes both columns cleanly so a run's selections can be reviewed and re-fed.

2.2 Weather data source

WEATHER_SOURCE (set by the **weather-data-toggle** control value) selects where degree-day data comes from. There are two modes:

Mode	Behavior
"sigma" (default)	Daily HDD/CDD are pulled from the SITE_DAILY_WEATHER_HISTORY element and summed to billing-period totals per site, joined on WeatherStation → ICAO Id. CSV HDD/CDD columns are used

Mode	Behavior
	as a row-level fallback wherever the Sigma lookup misses.
"csv"	HDD/CDD are taken directly from the input CSV columns. No Sigma weather query is performed.
Site exclusion If a site has no HDD/CDD value from either source on every one of its rows, the entire site is excluded from all calculations and listed in a warning. Partial coverage is allowed through — the weather-coverage gate handles sparse weather during model selection.	

2.3 Running the Engine

There are two entry points, selected automatically by how the script is invoked:

1. **In Sigma** — the workbook runs the script as a Python element. `_sigma_main()` reads the input element, assembles weather (if in Sigma mode), runs the analysis, and writes the output tables back to the workbook.
2. **Locally** — running the file directly (e.g. `python engine.py`) opens a file picker via `_local_main()`, analyzes the chosen CSV, and writes an Excel workbook with one sheet per output table. This is a holdover from older versions.

Control values configured in Sigma

Nearly every threshold is read at runtime from a Sigma control value (R^2 thresholds, CV-RMSE ceiling, baseline counts, outlier fences, seasonal-balance thresholds). These are configured in the workbook, not in the script, so behavior can be tuned without code changes. See Section 3 for the full list.

2.4 Outputs

The Engine produces eight tables. In Sigma they are written as named outputs; locally they become sheets in an Excel file.

Output table	Contents
<code>all_models</code>	One row per individual meter model and one row per site/fuel aggregate — distinguish them with the <code>Model Type</code> column (Individual / Aggregated). Renamed from <code>individual_models</code> , which stopped being accurate once aggregates were listed here too.
<code>aggregated_models</code>	The aggregate rows only, as a convenience view. Every aggregate appears in both tables (fit or not), so this is a strict subset of <code>all_models</code> — do not UNION the two or every aggregate double-counts.
<code>baseline_bills</code>	Every baseline bill, marked as used in the model or excluded as an outlier.
<code>reporting_bills</code>	Every reporting bill with predicted consumption, avoided usage, percent avoided, and bill-level avoided cost. This is the table the KPI tiles read.
<code>outliers</code>	Bills removed as baseline or reporting outliers, with the reason for each. Bills only — skipped whole sites are no longer listed here (they appear in <code>all_models</code> with <code>Optimization Mode = Skipped</code>).
<code>meter_weights</code>	Per-meter baseline-load weights for concurrent multi-meter sites, with coverage and risk flags.
<code>input_with_baseline</code>	The original input with clean Individual / Aggregated Selected Baseline columns stamped, plus corrected HDD/CDD, so selections can be reviewed or re-fed as a locked baseline.
<code>cost_only_review</code>	Sites whose cost-only (PUBLISHER / PASS_THROUGH) bills could not be consolidated automatically and were skipped from optimization pending a human decision.

2.5 Reading the results

This is the part most worth understanding. A model row carries three separate ideas that are easy to conflate: whether the model is statistically sound (quality gates), how its baseline was chosen (optimization mode), and whether it should be counted in the portfolio total (Recommended).

Quality gates

Each model is graded against **four** independent gates. **Model Passes Quality** is true only when all four pass.

Gate	Pass condition	Rating bands
Passes R²	$\geq$ service-specific threshold (gas and electricity differ)	Excellent ≥ 0.90 / Good ≥ 0.75 / Fair ≥ 0.60 / Poor below
Passes CV-RMSE	$\leq$ CV-RMSE threshold	Excellent < 10 / Good (ASHRAE Pass) ≤ 15 / Fair ≤ 25 / Poor above
Passes Weather Coverage	$\geq$ weather-coverage threshold (HDD-only for gas)	Excellent ≥ 90 / Good ≥ 75 / Fair ≥ 60 / Poor below
Passes Seasonal Balance	$\geq$ balance threshold (0.65 electric / 0.50 gas)	Excellent ≥ 0.90 / Good ≥ 0.75 / Fair ≥ 0.50 / Poor below
Seasonal balance vs. weather coverage — they are not the same		
Both come from the baseline, but they measure different things. Weather coverage is the width of your evidence — does the baseline span the HDD/CDD range the model must predict over? Seasonal balance is the density of it — is the baseline spread across the year, or is one season carrying the model? A baseline with 11 winter bills and 1 summer bill can span the full range (100% coverage) while resting the entire summer half of the model on a single bill (poor balance). Both gates must pass.		
A <i>rating</i> and its <i>verdict</i> can disagree for gas, by design: the pass threshold is 0.65 electric / 0.50 gas, so a “Fair” gas baseline passes where a “Fair” electric one does not — the same way the two R^2 thresholds differ.		

Quarterly Spread (%) is reported alongside these but is **not** part of **Model Passes Quality**. It reads the same four quarter counts as seasonal balance, summarized differently: spread penalizes a quarter with too *many* bills as hard as too few, while balance only cares about the thinnest quarter. Spread is used during selection (to filter and rank candidate baselines) and is relaxable; balance is the hard gate.

How the baseline was chosen — Optimization Mode

Optimization Mode	Meaning
Optimized	The optimizer searched bill combinations and picked the best one that cleared every selection gate.
Most Recent	No combination could clear the gates, so the model was fit on the most recent N clean bills with no search . Reviewable, but never Recommended — see below.
Locked	The baseline was read from a pre-selected (locked) set on this run rather than re-optimized.
Skipped	No model could be built at all — almost always too few clean baseline bills (fewer than the required 12 electric / 8 gas).

A companion column, **Seasonal Strategy**, describes what the baseline is (e.g. “Optimal (≥ 2 bills per quarter)” or “Most recent bills — no optimization”), re-derived from the bills each run. This is why a most-recent baseline that gets saved and re-uploaded shows **Optimization Mode = Locked** (true — this run read it from the file) while **Seasonal Strategy** still reports it was never optimized.

Why a model can pass R^2 / CV-RMSE / coverage and still not be recommended

The optimizer chooses a **representative baseline first** and only then fits the best model among those. R^2

and CV-RMSE are **not** selection filters — a combination with a great fit is never chosen if its baseline is seasonally lopsided. When no combination can pass, the site falls to the most-recent-N fallback. Such a model often fits well (good R^2 , good CV-RMSE, full coverage) but fails seasonal balance, because the site is genuinely missing a season. It is shown so an analyst can judge it, but it is not Recommended — and **not because of the fallback method**, but because of the real statistical gap, which **Recommendation Reason** names.

What to count — Recommended and Recommendation Reason

Recommended marks the single model per site/fuel that should flow into a portfolio total. It combines two things: the model passes quality, **and** it is the de-duplicated choice — at a site whose meters were aggregated, the passing aggregate is Recommended and its individual meters are not, so the site's load is counted once rather than several times. **Recommended = True** always implies **Model Passes Quality = True**; the reverse is not true (an aggregated site's individual meters can pass quality yet not be Recommended).

Recommendation Reason explains every row. A Recommended row reads “Recommended — no exclusions apply.” A non-Recommended row states the cause(s), joined with “ | ” — one of: aggregate preferred for this site/fuel, baseline not seasonally balanced (with the quarter counts), a specific quality miss (R^2 / CV-RMSE / coverage), or no model could be fit.

Savings status

Even a passing, Recommended model can have its savings excluded at the site level. **Savings Status** and **Savings Exclusion Flag** explain why. The main case: every clean reporting bill shows negative savings. This exclusion requires a **pattern** — at least two all-negative clean reporting bills — so a brand-new customer's single negative first bill does not drop the site out of reporting. Cost columns are always computed regardless; the status is a label, not a blanked value.

Filtering the KPI tiles

The savings tiles read reporting_bills. The recommended filter set, each doing a distinct job:

Filter	Job
Recommended = True	De-duplication (aggregate vs. its own meters) and model quality. The one filter that cannot be removed.
Individual Bill Status = 'Used in Savings'	Bill-level: drops reporting outliers and no-model bills. Tracks the outlier controls automatically.
Site-wide Savings Status = 'Included'	Site-level: drops sites whose savings are negative across two or more bills.

Two filters are best **removed** if present: **Model Passes Quality = True** (fully redundant behind **Recommended**) and a hardcoded % **Avoided Usage** range — the latter is a trap, because it duplicates the Reporting Outlier Ceiling / Floor controls and will silently disagree with the tool if those controls are changed.

Quarter labels lag the weather — read them by month range

Quarters are assigned by **Period Start Date**, but HDD/CDD accumulate across the whole period, so the label lags the weather by about a month. In this portfolio **Q4 (Oct-Dec starts) is the coldest quarter** (mean HDD ~178), above even Q1 (~163). Season names like “Fall” would mislead, which is why the bill-count columns state their month range: **Q1 Bills (Jan-Mar) ... Q4 Bills (Oct-Dec)**.

3. Configuration Reference

All tunable parameters are read from Sigma control values via `sigma.get_control_value(...)`. The tables below map each control key to its role in the Engine.

3.1 Quality thresholds

Control key	Constant	Role
<code>e-r2-thresh</code>	<code>ELEC_R2_THRESHOLD</code>	Minimum R^2 for an electricity model to pass.
<code>g-r2-thresh</code>	<code>GAS_R2_THRESHOLD</code>	Minimum R^2 for a gas model to pass (looser — gas fits HDD only).
<code>cvmse-thresh</code>	<code>CVRMSE_THRESHOLD</code>	Maximum CV-RMSE for any model to pass.
<code>weather-coverage-thresh</code>	<code>WEATHER_COVERAGE_THRESHOLD</code>	Minimum weather coverage percent to pass (measured on HDD only for gas).

3.2 Baseline selection

Control key	Constant	Role
<code>electric-baseline-count</code>	<code>ELEC_BASELINE_COUNT</code>	Number of baseline bills fitted for an electricity model (also the hard minimum below which the site is skipped).
<code>gas-baseline-count</code>	<code>GAS_BASELINE_COUNT</code>	Number of baseline bills fitted for a gas model (also the hard minimum).
<code>pref-min-per-quarter</code>	<code>PREFERRED_MIN_PER_QUARTER</code>	Minimum bills per calendar quarter the optimizer requires. Now the only per-quarter control.
<code>seasonal-balance-thresh</code>	<code>SEASONAL_BALANCE_THRESHOLD</code>	Min ratio of the weakest quarter to the mean quarter (electricity). At 0.65 with a 12-bill baseline this means ≥ 2 bills per quarter. A hard quality gate.
<code>gas-seasonal-balance-thresh</code>	<code>GAS_SEASONAL_BALANCE_THRESHOLD</code>	Balance ratio for gas (0.50). The idle-quarter exemption is what makes shapes like [2,2,1,3] reachable for heating-only meters.
<code>temporal-cv-ceiling</code>	<code>TEMPORAL_CV_CEILING</code>	Max Quarterly Spread ($\text{std}/\text{mean} \times 100$ of the quarterly counts). Reported as the “Quarterly Spread (%)” column. A relaxable selection filter, not a pass/fail gate.
<code>max-combos</code>	<code>MAX_COMBINATIONS</code>	Cap on the number of valid combinations kept per strategy.
<code>exhaustive-limit</code>	<code>EXHAUSTIVE_LIMIT</code>	Total-combination ceiling below which enumeration is exhaustive (else sampled).

Removed control: `abs-min-per-quarter`

The former `abs-min-per-quarter` / `MINIMUM_MIN_PER_QUARTER` control has been **removed**. Because a fresh baseline is always exactly the baseline count (12 or 8) across four quarters, the balance

Control key	Constant	Role
threshold already implies a flat per-quarter minimum that was stricter than that floor — so the “acceptable” fallback rung it fed could never accept anything the preferred pass hadn't. If the control still appears on the panel, it can be deleted; it has no effect.		

3.3 Multi-meter & weighting

Control key	Constant	Role
overlap-thresh	OVERLAP_THRESHOLD	Shared periods needed to treat two meters as concurrent.
weight-review-thresh	WEIGHT_REVIEW_THRESHOLD	Failing-meter load share above which an aggregate is flagged REVIEW.
weight-high-risk-review-thresh	WEIGHT_HIGH_RISK_THRESHOLD	Failing-meter load share above which an aggregate is flagged HIGH RISK.
low-coverage-thresh	LOW_COVERAGE_THRESHOLD	Weight below which a passing meter is flagged LOW COVERAGE.

3.4 Outliers, weather & elements

Control key	Constant	Role
reporting-outlier-floor	REPORTING_OUTLIER_LOW	Low fence (% avoided). Below it, consumption rose beyond the model's expectation.
reporting-outlier-ceil	REPORTING_OUTLIER_HIGH	High fence (% avoided). Above it, savings are implausibly large.
weather-data-toggle	WEATHER_SOURCE	Selects “sigma” or “csv” weather sourcing.
sigma-input-element	SIGMA_INPUT_ELEMENT	Name of the Sigma element holding billing data (asserted against the literal Input).
sigma-weather-element	SIGMA_WEATHER_ELEMENT	Name of the Sigma weather element (asserted against the literal Weather).
Important: element-name asserts Sigma requires the element name passed to <code>get_element()</code> to be a string literal. The script hard-codes “Input” and “Weather” in those calls and asserts that the matching control value equals the literal. If you rename an element, update both the control value and the literal together, or the run will fail the assert.		

3.5 Unit conversions

Consumption is converted to a common basis using the factors below (applied only when a Consumption Unit column is present).

Unit	Factor	Unit	Factor
kwh	1.0	therms / therm	1.0
mwh	1000.0	ccf	1.037
wh	0.001	mcf	10.37
		mmbtu	10.0

4. Architecture & Processing Pipeline

This section traces how data flows through the Engine from raw bills to output tables. It is the orienting map for the function reference in Sections 5 and 6.

4.1 High-level flow

1. **Entry.** Sigma or local entry point reads the input billing element/CSV.
2. **Preprocess.** Column names are normalized, dates/numerics parsed, charges cleaned, units converted, and locked-baseline state detected.
3. **Weather assembly (Sigma mode).** Daily weather is fetched, aggregated to billing periods, and merged into the bills with CSV fallback and a Weather Source tag.
4. **Flag inference.** Each bill is marked baseline or reporting relative to the commission date.
5. **Site exclusion.** Sites with no weather from any source are dropped.
6. **Per service / site / meter loop.** For each service type and site, meters are routed (single, sequential, or concurrent) and optimized.
7. **Optimize.** Candidate baseline combinations are filtered and scored; the best model is fitted and graded. Locked baselines bypass the search. If no combination clears the gates, the site falls back to the most recent N clean bills (no search).
8. **Aggregate & weight.** Concurrent multi-meter sites also get a weighted aggregate model with risk flags; only periods in which every pooled meter has a bill are summed. Locked aggregates reproduce from their own Aggregated Selected Baseline column.
9. **Build rows, de-duplicate & output.** Result, bill, outlier, and weight rows are assembled; a post-pass sets **Recommended** and **Recommendation Reason** (aggregate takes precedence over its meters). Eight output tables are written.

4.2 Module map

Group	Functions
Utilities	<code>_is_gas, _calc_cvrms, _n_choose_k, _random_combos, _safe_df</code>
Preprocessing	<code>preprocess, _convert_units, infer_flags</code>
Outlier detection	<code>detect_baseline_outliers, detect_reporting_outliers</code>
Model fitting	<code>fit_model, _weather_coverage, _seasonal_balance, _quarterly_spread, _idle_quarter, _seasonal_ok, _is_new_site, _combo_score</code>
Optimizer	<code>eval_combo, optimize, optimize_from_locked</code>
Multi-meter	<code>meters_concurrent, drop_incomplete_meter_periods, aggregate_site, calc_meter_weights, failing_weight_flag, low_coverage_flag</code>
Row builders	<code>build_result_row, compute_cost_cols, build_bill_row, build_baseline_rows, build_reporting_rows, build_outlier_row, build_meter_weight_rows, build_notes, backfill_original_weather, build_input_with_baseline, _recommendation_reason</code>
Weather assembly	<code>fetch_sigma_weather, aggregate_weather_to_periods, merge_weather</code>
Orchestration	<code>run_analysis, _sigma_main, _local_main</code>

5. Function Reference

Each function is documented with its purpose, key inputs/outputs, and noteworthy behavior. Functions are grouped to match the module map in Section 4.2.

5.1 Utilities

`_is_gas(service_type)`

Returns True when the service type string contains “gas” (case-insensitive). This single predicate drives the gas-vs-electric branching everywhere: number of regressors, R^2 threshold, baseline count, seasonal-balance threshold, and the idle-quarter exemption.

`_calc_cvrmse(actual, predicted)`

Computes the coefficient of variation of RMSE as a percentage of the mean actual value. Returns a sentinel of 999.0 when the mean is zero, so a degenerate series fails the CV-RMSE gate rather than dividing by zero.

`_n_choose_k(n, k)`

Integer combination count used to decide whether to enumerate baseline combinations exhaustively or sample them. Returns 0 when k exceeds n .

`_random_combos(n, k, count)`

Generator yielding up to count distinct random k -combinations of n items, used when the exhaustive space is too large.

`_safe_df(rows, schema_key)`

Builds a DataFrame from a list of row dicts, falling back to an empty frame with the correct schema columns when there are no rows. For the outliers schema it also reorders/filters columns to the canonical set.

5.2 Preprocessing

`preprocess(df)`

Normalizes the raw input into a working DataFrame. Strips and underscores column names, parses dates as UTC then drops the timezone, coerces HDD/CDD/Consumption to numeric, strips currency formatting from Total Charges, converts units, and normalizes the selected-baseline columns to “1” or blank.

- **Returns** a tuple: `(processed_df, df_original, has_locked)`. `df_original` keeps the original column formatting for the `input_with_baseline` output.
- Emits warnings about missing or NaN HDD/CDD columns.

`_convert_units(df)`

Multiplies Consumption by the factor for each Consumption Unit group (see Section 3.5). No-ops when the column is absent or the factor is 1.0.

`infer_flags(df)`

Adds Baseline_Bill and Reporting_Bill indicator columns by comparing each bill's period start date to the commission date. Raises ValueError if the commission date column is missing or entirely empty, since the baseline/reporting split cannot otherwise be made.

5.3 Outlier detection

`detect_baseline_outliers(bills)`

Flags baseline outliers in two ways: any negative consumption is always an outlier, and (when at least four non-negative bills exist) the standard $1.5 \times \text{IQR}$ fences identify too-low and too-high bills. Returns the cleaned bills and a frame of the removed rows with reasons.

`detect_reporting_outliers(model, bills, is_gas)`

Predicts each reporting bill with the fitted model and computes percent avoided. Bills outside the low/high percent-avoided fences (

`REPORTING_OUTLIER_LOW`

/

`HIGH`

) are removed as reporting outliers with a descriptive reason. Returns

`(clean, outliers)`

.

5.4 Model fitting & scoring

`fit_model(baseline, is_gas)`

Fits an ordinary least-squares linear regression. Gas uses HDD only; electricity uses HDD and CDD. Returns the model plus its in-sample R^2 and CV-RMSE.

`_weather_coverage(selected, all_baseline, is_gas=False)`

Measures how much of the full baseline HDD and CDD range the selected bills span, as a percentage. Returns

`(overall, hdd_coverage, cdd_coverage)`

.

- **Gas is HDD-only.** Because a gas model carries no CDD term, the headline (overall) number returns HDD coverage alone for gas; averaging in CDD coverage would blend in a dimension the model never uses and hide gaps. HDD and CDD are both still reported. Electricity averages the two.
- Returns 100% for an axis with zero range.

`_seasonal_balance(selected)`

Returns the weakest quarter's bill count $\div$ the mean quarter's count (1.00 = perfectly even, 0.00 = a season with no bills). This is the number behind

`Passes Seasonal Balance`

and the

`Seasonal Balance`

column.

`_quarterly_spread(selected, idle_q=None)`

Returns $\text{std} \div \text{mean} \times 100$ of the quarterly counts — the

`Quarterly Spread (%)`

column. Guards baseline shape (it separates [2,2,2,6] from [2,3,3,4], which seasonal balance scores identically).

Counts zero-bill quarters, and excludes the gas idle quarter so the same value drives the selection filter, the reported number, and the ranking.

`_idle_quarter(pool, is_gas)`

For gas, returns the site's warmest quarter — the one whose mean HDD is lowest by a clear margin — whose bills sit at $\text{HDD} \approx 0$ and carry no heating-slope information. Returns None for electricity (both HDD and CDD terms make every quarter informative). Derived per site, never hardcoded, so it adapts to each site's climate.

`_seasonal_ok(selected, min_per_q, balance_threshold, idle_q=None)`

Two-part seasonal gate: every quarter must meet the minimum bill count, and the weakest quarter must be at least

`balance_threshold`

of the mean. This gate is now part of

Model Passes Quality

on every path. The gas idle quarter (when supplied) is allowed one fewer bill and is dropped from the balance calculation. Gas uses

GAS_SEASONAL_BALANCE_THRESHOLD

.

`_is_new_site(baseline_bills)`

Returns True when the baseline spans less than 12 months (or has no parseable dates). New sites get a different scoring formula that omits the spread term.

`_combo_score(r2, temporal_cv, pct_avoided, reporting_count, max_reporting, is_new)`

Weighted composite score (higher is better) used to rank candidate baseline combinations.

- Normal: $0.45 \cdot R^2 + 0.30 \cdot \text{spread_inv} + 0.15 \cdot \text{pct} + 0.10 \cdot \text{rep_frac}$
- New site: $0.75 \cdot R^2 + 0.15 \cdot \text{pct} + 0.10 \cdot \text{rep_frac}$ (spread omitted)
- Components are normalized to roughly 0–1 before weighting; percent avoided is clamped to a [-10, 40] band. Selection also applies a hard precedence: any positive-savings passing model beats every non-positive one, and the model it beat is recorded for transparency.

5.5 Optimizer

`eval_combo(idx_list, baseline_pool, reporting_bills, all_baseline, r2_thresh, is_gas)`

Evaluates a single candidate baseline combination end-to-end: fits the model, computes weather coverage, seasonal balance and quarterly spread, cleans reporting outliers, predicts reporting consumption, and computes avoided usage/percent. Returns a rich result dict including the model, the selected and clean-reporting frames, the equation string, all quality metrics and pass/fail booleans (including seasonal balance), and quarterly bill counts.

`optimize(site_id, service_type, bills, baseline_count, r2_thresh)`

The core search. Splits bills into baseline and reporting, removes baseline outliers, and — if enough clean baseline bills remain — searches combinations for the best model.

- Enumerates combinations exhaustively when the count is at or below **EXHAUSTIVE_LIMIT**, otherwise samples randomly.
- Each candidate must pass the seasonal gate and weather-coverage minimum; non-new sites also face the quarterly-spread ceiling (relaxable). The idle-quarter exemption is derived once per site and applied throughout.
- Among valid combinations it prefers a passing positive-savings model, recording any higher-scoring non-positive model it passed over.
- **Ladder**. Runs a single preferred per-quarter strategy. If nothing passes, it falls back to the **most recent N clean bills with no search** (**Optimization Mode = Most Recent**), which fails the seasonal gate on its own statistics and so is never Recommended. The former “looser minimum-per-quarter” retry was removed as a no-op. Returns (**result, baseline_outliers, reporting_outliers, note**).

`optimize_from_locked(site_id, service_type, bills, r2_thresh)`

Reconstructs a model directly from pre-selected baseline bills without searching. Used on re-runs so analysts can lock a chosen baseline.

- **Coverage is now measured, not assumed**. It rebuilds a clean baseline pool and compares the locked selection against it (a prior version compared the selection to itself and reported a fake 100%).
- Runs the seasonal check and populates Passes Seasonal Balance on locked rows.
- Re-derives whether the locked set is exactly the most recent N bills, so **Seasonal Strategy** reports “Most recent bills — no optimization” when true even after a save/re-upload. Labels the strategy “Locked (pre-selected)” otherwise.

5.6 Multi-meter handling

meters_concurrent(site_data)

Returns True if any two meters at a site share at least

OVERLAP_THRESHOLD

billing-period start dates, indicating they ran at the same time (rather than one replacing another).

drop_incomplete_meter_periods(site_data, meters)

Before a concurrent site is aggregated, drops any billing period that is missing a bill from one or more of the pooled meters, so the summed total always reflects the full set of meters. Prevents under-counted periods from corrupting the aggregate regression (which had produced physically impossible coefficient signs). Returns the trimmed data and a coverage note.

aggregate_site(site_data, agg_lock_col=None)

Sums consumption and charges across meters per billing period to produce a single AGGREGATED meter timeline, carrying through weather, flags, and commission date. When an aggregate lock column is supplied, it rebuilds the aggregate baseline from that column so a locked aggregate reproduces exactly.

calc_meter_weights(site_data, meter_nums)

Computes each meter's share of total baseline consumption. When total baseline load is zero, weights are split evenly. These weights drive coverage and risk flags.

failing_weight_flag(meter_weights, ind_results)

Sums the baseline-load weight of meters whose individual models fail quality. Returns HIGH RISK above the high-risk threshold, REVIEW above the review threshold, otherwise OK — with an explanatory note naming the failing meters and their combined share.

low_coverage_flag(weight, passes_quality)

Flags a passing meter whose weight is below

LOW_COVERAGE_THRESHOLD

as LOW COVERAGE, indicating it verifies only a small slice of site load.

5.7 Row builders

build_result_row(...)

Assembles a single model row for

all_models

/

aggregated_models

: ratings for R^2 , CV-RMSE, weather coverage and

seasonal balance

; the four pass/fail booleans; quarterly counts (labeled by month range) and quarterly spread; avoided usage and percent; notes; the cost columns from

compute_cost_cols

; and placeholders for

Recommended

/

Recommendation Reason

(set later in the de-duplication pass).

compute_cost_cols(res)

Derives the dollar figures: a measured rate (charges ÷ consumption), modeled and actual cost, avoided cost, and average monthly avoided cost.

- **Always computes** the cost columns; **Savings Status** is a label rather than a blanked value.
- Returns an excluded status when there is no model or all reporting bills were removed as outliers, or when every clean reporting bill shows negative savings — the latter only on a **pattern** of at least **MIN_REPORTING_BILLS_FOR_NEGATIVE_EXCLUSION** (2) all-negative bills, so a single negative first bill is kept.

build_bill_row(r, site, meter_num, service_type, period_type, status)

Creates a base bill row (used by both baseline and reporting builders) with identifiers, dates, weather, consumption, charges, and placeholders for the prediction columns.

build_baseline_rows(res, site, meter_num, service_type)

Emits one row per selected baseline bill (marked Used in Model, with the appropriate Individual / Aggregated Selected Baseline flag) plus rows for baseline outliers labeled with their exclusion reason.

build_reporting_rows(res, site, meter_num, service_type, model_passes, savings_status)

Emits one row per reporting bill — clean bills as Used in Savings and outliers as Excluded — each with predicted consumption, avoided usage, percent avoided, bill avoided cost, the model's quality metrics, and (via the later pass)

Recommended

/

Recommendation Reason

.

build_outlier_row(o, site, meter_num, service_type)

Normalizes an outlier record into the canonical outliers schema. Only bill-level outliers are emitted; skipped whole sites are not written here (they appear in

all_models

as Skipped).

build_meter_weight_rows(...)

Builds the

meter_weights

rows for a concurrent site: each meter's baseline consumption, weight percent, weight source, individual-model quality, and coverage/aggregate flags.

build_notes(ind_results, agg_res, meter_count)

Composes a human-readable summary of how many individual meters pass vs. the aggregate, with average individual and aggregate R² and savings.

backfill_original_weather(df_original, enriched)

Joins the enriched period HDD/CDD back onto the original input rows by (Site, Period Start, Period End) so

input_with_baseline

carries correct degree-days (the original snapshot was taken before the weather merge). Normalizes timezone-aware and naive dates before joining.

build_input_with_baseline(df_original, selected_baseline_keys, aggregated_baseline_keys=None)

Returns the original input with clean

Individual Selected Baseline

and

Aggregated Selected Baseline

columns stamped. Preserves previously locked “1” marks on re-upload and adds this run's selections, matching on (site, meter, period start); aggregate selections are matched via the all-meters rule.

`_recommendation_reason(row, agg_preferred_here)`

Builds the

Recommendation Reason

string. A Recommended row returns “Recommended — no exclusions apply” (never blank, since a blank is unfilterable in Sigma). A non-Recommended row states its cause(s), joined with “ | ”: aggregate preferred, baseline not seasonally balanced (with quarter counts), a specific quality miss, or no model fit — always the statistical defect, never the selection method.

6. Sigma Integration

This section covers the functions that connect the Engine to the M&VEngine_MASTER workbook: weather assembly, the orchestrator, and the entry points.

6.1 Weather assembly

`fetch_sigma_weather()`

Reads the weather element (literal name “Weather”, asserted against

`SIGMA_WEATHER_ELEMENT`

) and returns daily rows. Sigma returns opaque column IDs, so the function renames the four needed columns (Icao Id, Date Valid Std, Hdd Lt 65 F, Cdd Gt 65 F) via a fixed ID map, normalizes types, and returns an empty frame (triggering full CSV fallback) if the element is missing, empty, or lacks expected columns after rename.

`aggregate_weather_to_periods(daily_df, billing_df)`

For each unique (site, station, period) key, sums daily HDD/CDD where the daily date falls within the period window [start, end). Joins on WeatherStation → Icao Id. Periods with no daily coverage produce NaN, and the result is keyed on Site_Name so it can merge back to billing.

`merge_weather(billing_df, weather_df)`

Merges Sigma-sourced HDD/CDD into the bills on (site, period start).

- Resolution order per row: Sigma value when present, else the CSV value, else NaN.
- Adds a Weather Source column tagging each row as Sigma, CSV, or No Data, and prints a resolution summary plus a warning for any No Data rows.

6.2 Orchestration & entry points

`run_analysis(df, already_preprocessed, df_original, has_locked)`

The main driver. Optionally preprocesses and infers flags, excludes weather-less sites, then loops over each service type and site. It routes meters (single, sequential aggregate, or concurrent per-meter), optimizes or reconstructs each model, builds all row sets, computes meter weights and the aggregate model for concurrent sites, runs the de-duplication pass that sets

`Recommended`

/

`Recommendation Reason`

, and returns the dictionary of eight output DataFrames (with a run timestamp on

`all_models`

).

- **Meter routing:** no meter → single model; multiple non-concurrent meters → one combined (sequential) timeline; multiple concurrent meters → per-meter models plus a weighted aggregate.

`_sigma_main()`

Sigma entry point. In CSV mode it reads the raw upload element and runs the analysis directly. In Sigma mode it reads the Input element, preprocesses, fetches and merges weather, runs the analysis with the preprocessed flag set, and writes all eight outputs back via

`sigma.output(...)`

(including

`all_models`

, renamed from

`individual_models`

, and

`cost_only_review`

).

`_local_main()`

Local entry point. Opens a file dialog, reads the chosen CSV, runs the analysis, and writes an Excel workbook with one sheet per output table next to the source file. Invoked only when the script is run as

`__main__`

; otherwise

`_sigma_main()`

runs.

Maintainer notes

Renamed output. The `individual_models` output is now `all_models`. The Sigma element that consumed it must be rebound, and any pivot referencing the old “Temporal CV” column should point at “Quarterly Spread (%)” (or at the “Seasonal Balance” column, with its color scale set green-at-high).

Weather column IDs. The opaque-ID map in `fetch_sigma_weather()` is tied to a specific Weather element schema. If that element is rebuilt, the IDs change and the rename silently misses — the function then falls back to CSV for all rows. Re-capture the IDs when the element changes.

Element renames. Keep the `sigma-input-element` / `sigma-weather-element` control values in sync with the string literals in the `get_element()` calls, or the runtime asserts will fail.